

Random vs Stratified Sampling in R: Building a Fair Train-Test Split

Sampling in Data Mining

Sampling is the process of selecting a reduced subset capable of faithfully representing the complete dataset. Through this technique, mining algorithms can operate with potentially sub-linear complexity relative to data size, making it feasible to analyze large volumes of information without compromising the quality of results. A good sample must preserve the key statistical properties of the original data — including its distribution, variability, and class balance — so that conclusions drawn from it generalize to the whole.

Fundamental Principle

Select a representative subset that preserves the essential statistical characteristics of the original dataset, ensuring that conclusions drawn from the sample are generalizable to the whole

Sampling Rate Trade-off

Choosing the right sample size involves balancing three competing factors: computational cost, estimation accuracy, and representativeness — a sample too small introduces bias, while one too large defeats the purpose of sampling

Pipeline Position

Sampling must occur as the very first step in any ML pipeline — before any preprocessing, transformation, or normalization — to prevent data leakage between training and test sets and ensure unbiased model evaluation

📌 **Important note:** It is worth emphasizing that sampling does not necessarily reduce I/O operations in databases, since access is performed page by page. The efficiency gain manifests primarily in the algorithmic processing that follows data reading.

Types of Sampling

Simple Random Sampling

Each item in the dataset has exactly the same probability of being selected, ensuring a truly unbiased choice among all available elements.

Sampling With Replacement (SRSWR)

Unlike the previous approach, the selected object remains in the population after each draw. Thus, the same element may appear multiple times throughout the sample.

Sampling Without Replacement (SRSWOR)

Once selected, each object is immediately removed from the population. This way, no element can be chosen more than once, ensuring samples without repetition.

Cluster Sampling

The population is divided into natural groups (clusters); entire clusters are selected at random. Efficient when natural groupings exist, but may introduce higher variance than stratified approaches.

Sampling: With or Without Replacement

The replacement strategy defines whether a selected instance can be drawn again. This choice has direct implications for sample diversity, statistical independence, and the behavior of resampling methods like bootstrap.

SRSWOR

(Simple Random Sampling Without Replacement)

In this method, each selected element is removed from the candidate pool. As a result, the population progressively decreases with each selection, which changes the probabilities of subsequent choices.

Advantages:

- Ensures greater diversity in the sample
- Avoids duplication of elements
- Especially suitable for smaller sample sizes
- Used by default in train/test splits — each instance appears in exactly one partition

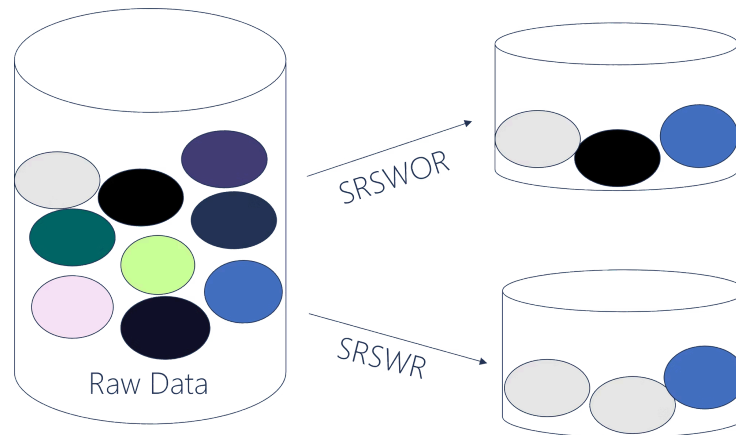
SRSWR

(Simple Random Sampling With Replacement)

In this case, elements return to the pool after each selection, keeping probabilities constant throughout the process. This way, each choice is completely independent of the previous ones.

Advantages:

- Greater mathematical simplicity
- Ensures statistical independence between selections
- Particularly useful in bootstrap techniques
- Foundation of bootstrap resampling — allows estimating sampling distributions empirically



❏ In the daltoolbox, both `sample_random()` and `sample_stratified()` operate without replacement by default, ensuring each instance appears in only one partition per split.

Cluster Sampling

Cluster sampling exploits the natural grouping structure of data. Instead of sampling individual instances, entire groups (clusters) are selected at random — making it especially practical when data is naturally organized into geographic regions, time windows, experimental batches, or other meaningful partitions.

How It Works

- Divide the population into mutually exclusive clusters
- Randomly select a subset of clusters
- Include all instances from selected clusters in the sample

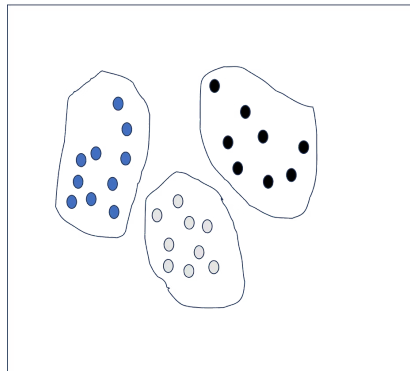
When to Use It

- Efficient when natural groupings exist and collecting data per cluster is cheaper than individual sampling
- Common in surveys, sensor networks, and batch experiments

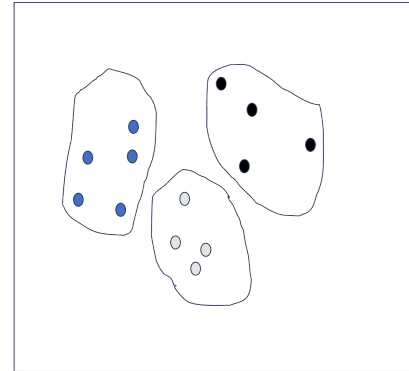
Limitations

- If clusters are internally homogeneous but differ from each other, variance increases
- Results may be less precise than stratified sampling for the same sample size

Raw Data



Cluster/Stratified Sample



Practical Sampling Examples

Below, we visualize different sampling approaches applied to the same dataset, allowing us to observe how each method selects its elements in a distinct way and with varying impacts on the representativeness of the sample:

2/3

	setosa	versicolor	virginica
dataset	50	50	50
random sample	42	41	37
stratified sample	40	40	40

2/3

Higher Sampling Rate

Captures approximately two thirds of the original population

1/3

	setosa	versicolor	virginica
random sample	8	11	11
stratified sample	10	10	10

1/3

Lower Sampling Rate

Selects around one third of the available data

Defining the ideal sampling rate involves balancing multiple factors: available computational resources, the inherent variability of the data, the precision required by the analysis, and the tolerable processing time. In general, larger samples tend to produce more accurate estimates — however, they also demand greater resource consumption and execution time.

Random Sampling – Theory

Random sampling is a probabilistic selection method in which **each instance of the dataset has an equal probability of being chosen**, with no preferential criterion applied. The selection process is entirely stochastic, meaning that every element competes on equal terms for inclusion in the sample.

In machine learning workflows, random sampling is commonly applied in two scenarios:

- **Train/test splits:** the dataset is partitioned into a training set and a test set by randomly allocating instances, typically in a fixed ratio (e.g., 70/30 or 80/20).
- **K-fold cross-validation:** the dataset is randomly shuffled and divided into k equally-sized folds, each serving as the test set once while the remaining folds form the training set.

When to use it: random sampling is most appropriate for **balanced and homogeneous datasets**, where the class distribution is relatively uniform and preserving class proportions is not a critical concern.

A key practical consideration is **reproducibility**: because the selection is stochastic, results may vary between runs. Fixing a random seed via `set.seed()` ensures that the same partition is produced every time, making experiments reproducible and comparable.

Equal Probability

Every instance in the dataset has an identical probability of being selected. No weighting, stratification, or preferential criterion is applied during the sampling process.

No Guarantee of Proportions

Class distributions are **not guaranteed** to be preserved between training and test sets. This can lead to imbalanced splits and introduce **evaluation bias**, especially in datasets with minority classes.

Reproducibility with `set.seed`

Because sampling is stochastic, results may differ across runs. Setting a fixed random seed via `set.seed()` ensures consistent, reproducible partitions across experiments.

Random Sampling – sample_random

```
# Loading the package
library(daltoolbox)

iris <- datasets::iris
table(iris$Species)
# setosa versicolor virginica
#  50    50    50

# Random train/test split
tt <- train_test(sample_random(), iris)
print(table(tt$train$Species))
# setosa versicolor virginica
#  33    42    45
```

```
print(table(tt$test$Species))
# setosa versicolor virginica
#  17    8    5

# Random K-folds (4 folds)
sample <- sample_random()
folds <- k_fold(sample, iris, 4)
tbl <- NULL
for (f in folds) {
  tbl <- rbind(tbl, table(f$Species))
}
print(tbl)
```

Step by step

- Load the `iris` dataset and verify the original class distribution before any split
- Then, use `train_test(sample_random(), iris)` to split the data into train and test by random draw
- Note that class proportions are **not guaranteed** — they vary with each run, which may introduce bias in the evaluation
- Finally, apply `k_fold` to create 4 partitions; the distribution per fold may also vary between runs

📌 **Key concept:** Random sampling preserves proportions only *on average*. Therefore, comparing models on different partitions without fixing the seed (`set.seed`) can compromise reproducibility and the validity of the evaluation.

Stratified Sampling – Theory

Stratified sampling divides the dataset into homogeneous groups called strata (e.g., classes in a classification problem), and samples are drawn from each stratum proportionally to its representation in the full dataset.

This guarantees that the class distribution is faithfully preserved in both training and test sets — unlike random sampling, which offers no such guarantee.

Applied to train/test splits: each class contributes proportionally to both partitions (e.g., 80% train / 20% test per class).

Applied to k-fold cross-validation: each fold receives approximately the same number of instances per class, ensuring balanced evaluation across all folds.

When to use it: strongly recommended for imbalanced datasets, where minority classes could be underrepresented or entirely absent in some folds under random sampling.

Proportional Representation

Each stratum (class) contributes proportionally to every partition. The original class distribution is preserved in both training and test sets.

Reduced Evaluation Variance

By ensuring balanced class counts across folds, stratification reduces variance in performance metrics, leading to more reliable model comparisons.

Essential for Imbalanced Data

When classes are imbalanced, random sampling risks producing folds with very few or zero instances of minority classes. Stratification prevents this, ensuring every fold is evaluable.

📖 **Reference:** Kohavi, R. (1995). A study of cross-validation and bootstrap for accuracy estimation and model selection. *IJCAI*, 14(2), 1137–1145.

Stratified Sampling – sample_stratified

```
library(daltoolbox)

iris <- datasets::iris
# Stratified train/test split
tt <- train_test(
  sample_stratified("Species"), iris
)
print(table(tt$train$Species))
# setosa versicolor virginica
# 40 40 40

print(table(tt$test$Species))
# setosa versicolor virginica
# 10 10 10
```

```
# Stratified K-folds (4 folds)
sample <- sample_stratified("Species")
folds <- k_fold(sample, iris, 4)
tbl <- NULL
for (f in folds) {
  tbl <- rbind(tbl, table(f$Species))
}
print(tbl)
# setosa versicolor virginica
# 13 13 13
# 13 13 13
# 12 12 12
# 12 12 12
```

Step by step

- Start by passing the target column directly to `sample_stratified("Species")`
- This ensures that the train/test split guarantees an **exact proportion** per class in each partition — unlike purely random sampling
- The same logic applies to the folds: each one receives approximately 13 or 12 instances per class, maintaining balance throughout cross-validation
- For these reasons, stratification is the most recommended choice for problems with imbalanced classes

❏ **Key concept:** Stratification ensures that each partition faithfully represents the original distribution of the dataset — an essential condition for reliable evaluation, especially when classes are imbalanced. (Kohavi, 1995)



Acknowledgements



Data Analytics Lab

dal.global@grupo.cefet-rj.br | <https://eic.cefet-rj.br/dal>

